

Skill Session 21

Task 1

To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components

```
j_rakshitha@RAKSHITHA:~$ nano session21_task1_modules.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session21_task1_modules.c -o session21_t
ask1_modules
j_rakshitha@RAKSHITHA:~$ ./session21_task1_modules
Practical Skill Session 21 - Task 1
=====
Module integration shell started.
PID: 1890

Available commands:
  run <command>  - execute a command
  pwd            - display current directory
  help          - display this help
  exit          - quit shell

module-shell> pwd
/home/j_rakshitha
[Shell] Command exited with status 0.

module-shell> run echo Hello_OSSP
Hello_OSSP
[Shell] Command exited with status 0.

module-shell> run ls /this_directory_does_not_exist
ls: cannot access '/this_directory_does_not_exist': No such file or directory
[Shell] Command exited with status 2.

module-shell> run definitely_not_a_command
sh: 1: definitely_not_a_command: not found
[Shell] Command exited with status 127.

module-shell> exit

Shell exiting safely.
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session21_task1_modules.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <errno.h>

#define MAX_INPUT 100

void execute_command(char *command)
{
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        execl("/bin/sh", "sh", "-c", command, (char *)NULL);

        perror("execl");
        exit(127);
    }

    int status;

    while (waitpid(pid, &status, 0) == -1) {
        if (errno == EINTR)
            continue;

        perror("waitpid");
        return;
    }

    if (WIFEXITED(status)) {
        printf("[Shell] Command exited with status %d.\n",
            WEXITSTATUS(status));
    }
    else if (WIFSIGNALED(status)) {
        printf("[Shell] Command terminated by signal %d.\n",
            WTERMSIG(status));
    }
}

void show_help(void)
{
    printf("\nAvailable commands:\n");
    printf("  run <command>  - execute a command\n");
    printf("  pwd            - display current directory\n");
    printf("  help          - display this help\n");
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_/ Go To Line M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session21_task1_modules.c
printf("  exit          - quit shell\n");
}

int main(void)
{
    char input[MAX_INPUT];

    printf("Practical Skill Session 21 - Task 1\n");
    printf("=====\n");
    printf("Module integration shell started.\n");
    printf("PID: %d\n", getpid());

    show_help();

    while (1) {
        printf("\nmodule-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0) {
            break;
        }

        if (strcmp(input, "help") == 0) {
            show_help();
            continue;
        }

        if (strcmp(input, "pwd") == 0) {
            execute_command("pwd");
            continue;
        }

        if (strncmp(input, "run ", 4) == 0) {
            if (strlen(input) == 4) {
                printf("[Shell] Error: command cannot be empty.\n");
                continue;
            }

            execute_command(input + 4);
            continue;
        }

        if (strlen(input) == 0)
            continue;

        printf("[Shell] Unknown command: %s\n", input);
    }

    printf("\nShell exiting safely.\n");
    return 0;
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
}

printf("[Shell] Unknown command: %s\n", input);

printf("\nShell exiting safely.\n");

return 0;
}
```

Task 2

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

```
j_rakshitha@RAKSHITHA:~$ nano session21_task2_errors.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session21_task2_errors.c -o session21_ta
sk2_errors
j_rakshitha@RAKSHITHA:~$ ./session21_task2_errors
Practical Skill Session 21 - Task 2
=====
Runtime error handling shell started.
PID: 1960

Commands:
  run <command>  - execute a command
  log            - display error log
  help          - display help
  exit          - quit shell

error-shell> run ls /this_directory_does_not_exist
ls: cannot access '/this_directory_does_not_exist': No such file or directory
[Shell] Command exited with status 2.

error-shell> log

===== Error Log =====
[ERROR] Command failed: 'ls /this_directory_does_not_exist' (exit status 2)
=====

error-shell> badcommand
[Shell] Error: invalid syntax or unknown command.

error-shell> log

===== Error Log =====
[ERROR] Command failed: 'ls /this_directory_does_not_exist' (exit status 2)
[ERROR] Invalid syntax or unknown command
=====

error-shell> exit

Shell exiting safely.
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session21_task2_errors.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <sys/wait.h>

#define LOG_FILE "session21_errors.log"
#define MAX_INPUT 200

void log_error(const char *message)
{
    FILE *log = fopen(LOG_FILE, "a");

    if (log == NULL) {
        perror("[Logger] Cannot open log file");
        return;
    }

    fprintf(log, "[ERROR] %s\n", message);
    fclose(log);
}

void execute_command(const char *command)
{
    pid_t pid = fork();

    if (pid < 0) {
        perror("[Shell] fork");
        log_error("fork failed");
        return;
    }

    if (pid == 0) {
        execl("/bin/sh", "sh", "-c", command, (char *)NULL);

        perror("[Child] execl");
        _exit(127);
    }

    int status;

    if (waitpid(pid, &status, 0) == -1) {
        perror("[Shell] waitpid");
        log_error("waitpid failed");
        return;
    }

    if (WIFEXITED(status)) {
        int exit_status = WEXITSTATUS(status);
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_/ Go To Line M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session21_task2_errors.c
    int exit_status = WEXITSTATUS(status);

    printf("[Shell] Command exited with status %d.\n",
           exit_status);

    if (exit_status != 0) {
        char message[MAX_INPUT];

        snprintf(message, sizeof(message),
                 "Command failed: '%s' (exit status %d)",
                 command, exit_status);

        log_error(message);
    }
}
else if (WIFSIGNALED(status)) {
    int signal_number = WTERMSIG(status);

    printf("[Shell] Command terminated by signal %d.\n",
           signal_number);

    char message[MAX_INPUT];

    snprintf(message, sizeof(message),
             "Command terminated by signal %d: '%s'",
             signal_number, command);

    log_error(message);
}
}

void show_log(void)
{
    FILE *log = fopen(LOG_FILE, "r");

    if (log == NULL) {
        if (errno == ENOENT) {
            printf("[Logger] No log file exists yet.\n");
        }
        else {
            perror("[Logger] fopen");
        }
        return;
    }

    char line[MAX_INPUT];

    printf("\n==== Error Log ==== \n");

    while (fgets(line, sizeof(line), log) != NULL) {
        printf("%s", line);
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session21_task2_errors.c
    printf("%s", line);
}

printf("=====\n");

fclose(log);
}

void show_help(void)
{
    printf("\nCommands:\n");
    printf("  run <command>  - execute a command\n");
    printf("  log            - display error log\n");
    printf("  help          - display help\n");
    printf("  exit          - quit shell\n");
}

int main(void)
{
    char input[MAX_INPUT];

    printf("Practical Skill Session 21 - Task 2\n");
    printf("=====\n");
    printf("Runtime error handling shell started.\n");
    printf("PID: %d\n", getpid());

    show_help();

    while (1) {
        printf("\nerror-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0) {
            break;
        }

        if (strcmp(input, "help") == 0) {
            show_help();
            continue;
        }

        if (strcmp(input, "log") == 0) {
            show_log();
            continue;
        }
    }
}
```

```
        if (strncmp(input, "run ", 4) == 0) {
            if (strlen(input) == 4) {
                printf("[Shell] Error: command cannot be empty.\n");
                log_error("Empty command supplied");
                continue;
            }

            execute_command(input + 4);
            continue;
        }

        if (strlen(input) == 0)
            continue;

        printf("[Shell] Error: invalid syntax or unknown command.\n");
        log_error("Invalid syntax or unknown command");
    }

    printf("\nShell exiting safely.\n");

    return 0;
}
```

Skill Session 22

Task 1

To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

```
j_rakshitha@RAKSHITHA: ~$ nano session22_task1_valgrind.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session22_task1_valgrind.c -o session22_task1_valgrind
j_rakshitha@RAKSHITHA:~$ valgrind --leak-check=full ./session22_task1_valgrind

==679== Memcheck, a memory error detector
==679== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==679== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==679== Command: ./session22_task1_valgrind
==679==
Practical Skill Session 22 - Task 1
=====
Allocated buffer: OSSP memory testing
Allocated second buffer: Valgrind
Second buffer freed successfully.
==679==
==679== HEAP SUMMARY:
==679==   in use at exit: 100 bytes in 1 blocks
==679==   total heap usage: 3 allocs, 2 frees, 1,144 bytes allocated
==679==
==679== 100 bytes in 1 blocks are definitely lost in loss record 1 of 1
==679==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==679==    by 0x400121C: main (in /home/j_rakshitha/session22_task1_valgrind)
==679==
==679== LEAK SUMMARY:
==679==    definitely lost: 100 bytes in 1 blocks
==679==    indirectly lost: 0 bytes in 0 blocks
==679==    possibly lost: 0 bytes in 0 blocks
==679==    still reachable: 0 bytes in 0 blocks
==679==    suppressed: 0 bytes in 0 blocks
==679==
==679== For lists of detected and suppressed errors, rerun with: -s
==679== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

```
j_rakshitha@RAKSHITHA:~$ nano session22_task1_valgrind.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session22_task1_valgrind.c -o session22_task1_valgrind
j_rakshitha@RAKSHITHA:~$ valgrind --leak-check=full ./session22_task1_valgrind

==723== Memcheck, a memory error detector
==723== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==723== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==723== Command: ./session22_task1_valgrind
==723==
Practical Skill Session 22 - Task 1
=====
Allocated buffer: OSSP memory testing
Allocated second buffer: Valgrind
Second buffer freed successfully.
First buffer freed successfully.
==723==
==723== HEAP SUMMARY:
==723==       in use at exit: 0 bytes in 0 blocks
==723==   total heap usage: 3 allocs, 3 frees, 1,144 bytes allocated
==723==
==723== All heap blocks were freed -- no leaks are possible
==723==
==723== For lists of detected and suppressed errors, rerun with: -s
==723== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session22_task1_valgrind.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    printf("Practical Skill Session 22 - Task 1\n");
    printf("=====\n");

    char *buffer = malloc(100);

    if (buffer == NULL) {
        perror("malloc");
        return 1;
    }

    strcpy(buffer, "OSSP memory testing");

    printf("Allocated buffer: %s\n", buffer);

    char *second_buffer = calloc(20, sizeof(char));

    if (second_buffer == NULL) {
        perror("calloc");
        free(buffer);
        return 1;
    }

    strcpy(second_buffer, "Valgrind");

    printf("Allocated second buffer: %s\n", second_buffer);

    free(second_buffer);

    printf("Second buffer freed successfully.\n");

    free(buffer);

    printf("First buffer freed successfully.\n");

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Task 2

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

```
j_rakshitha@RAKSHITHA:~$ nano session22_task2_tests.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session22_task2_tests.c -o session22_task2_tests
j_rakshitha@RAKSHITHA:~$ ./session22_task2_tests hello
TEST PASSED: Hello input recognized.
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session22_task2_tests.c
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: %s <test_input>\n", argv[0]);
        return 1;
    }

    if (strcmp(argv[1], "hello") == 0)
    {
        printf("TEST PASSED: Hello input recognized.\n");
        return 0;
    }

    if (strcmp(argv[1], "ossp") == 0)
    {
        printf("TEST PASSED: OSSP input recognized.\n");
        return 0;
    }

    printf("TEST FAILED: Unknown input '%s'.\n", argv[1]);
    return 2;
}
```

```
j_rakshitha@RAKSHITHA:~$ nano session22_task2_test.sh
j_rakshitha@RAKSHITHA:~$ chmod +x session22_task2_test.sh
j_rakshitha@RAKSHITHA:~$ ./session22_task2_test.sh
=====
Session 22 Task 2 - Automated Tests
=====

Test 1: hello
TEST PASSED: Hello input recognized.

Test 2: ossp
TEST PASSED: OSSP input recognized.

Test 3: invalid input
TEST FAILED: Unknown input 'invalid'.

Test 4: no input
Usage: ./session22_task2_tests <test_input>

=====
Testing completed
=====
```

```
j_rakshitha@RAKSHITHA: ~  
GNU nano 8.7.1 session22_task2_test.sh  
#!/bin/bash  
  
echo "=====  
echo "Session 22 Task 2 - Automated Tests"  
echo "=====  
  
echo  
echo "Test 1: hello"  
./session22_task2_tests hello  
  
echo  
echo "Test 2: ossp"  
./session22_task2_tests ossp  
  
echo  
echo "Test 3: invalid input"  
./session22_task2_tests invalid  
  
echo  
echo "Test 4: no input"  
./session22_task2_tests  
  
echo  
echo "=====  
echo "Testing completed"  
echo "=====
```

```
j_rakshitha@RAKSHITHA:~$ nano session22_task2_test.sh  
j_rakshitha@RAKSHITHA:~$ ./session22_task2_test.sh  
=====  
Session 22 Task 2 - Test Report  
=====  
  
Test 1: hello  
RESULT: PASS  
  
Test 2: ossp  
RESULT: PASS  
  
Test 3: invalid input  
RESULT: PASS  
  
=====  
TEST SUMMARY  
=====  
Passed tests: 3  
Failed tests: 0  
Total tests : 3  
Overall result: ALL TESTS PASSED  
=====  
j_rakshitha@RAKSHITHA:~$
```

j_rakshitha@RAKSHITHA: ~

+

▼

GNU nano 8.7.1

session22_task2_test.sh

#!/bin/bash

```
echo "====="
echo "Session 22 Task 2 - Test Report"
echo "====="
```

```
passed=0
failed=0
```

```
echo
echo "Test 1: hello"
output=$(./session22_task2_tests hello)
```

```
if echo "$output" | grep -q "TEST PASSED"; then
    echo "RESULT: PASS"
    passed=$((passed + 1))
else
    echo "RESULT: FAIL"
    failed=$((failed + 1))
fi
```

```
echo
echo "Test 2: ossp"
output=$(./session22_task2_tests ossp)
```

```
if echo "$output" | grep -q "TEST PASSED"; then
    echo "RESULT: PASS"
    passed=$((passed + 1))
else
    echo "RESULT: FAIL"
    failed=$((failed + 1))
fi
```

```
echo
echo "Test 3: invalid input"
output=$(./session22_task2_tests invalid)
```

```
if echo "$output" | grep -q "TEST FAILED"; then
    echo "RESULT: PASS"
    passed=$((passed + 1))
else
    echo "RESULT: FAIL"
    failed=$((failed + 1))
fi
```

```
echo
echo "=====
echo "TEST SUMMARY"
echo "=====
echo "Passed tests: $passed"
echo "Failed tests: $failed"
echo "Total tests : $((passed + failed))"

if [ "$failed" -eq 0 ]; then
    echo "Overall result: ALL TESTS PASSED"
else
    echo "Overall result: SOME TESTS FAILED"
fi

echo "=====
|
```